# Homework 5

All the problems are based on the two structures, NODE and ROOT, defined on the right. Note that you CANNOT make any changes to these structures. Answers to the first 3 are posted on GLUE, answers to 4-6 and 17-18 will be discussed in class. Solve at least 4 other problems and name your answers as 5_x.c (where x is the number of the problems you choose) and submit them.

```
typedef struct node
{ int * data;
  struct node * next;
} NODE;

typedef struct
{ NODE * head;
} ROOT;
```

1. Write a complete program to create 10 NODEs with random numbers between 10 and 20 (including both) as their data values, connect these NODEs with a linked list, and define a ROOT pointer that points to the first NODE in the linked list.
2. Implement a function that takes a ROOT pointer as input and prints out the data values in the linked list one by one.
3. Implement a function that takes a ROOT pointer as input and prints out the largest value in the linked list.

MeshSec Lab



# Homework 5 (* indicates the difficulty level.)

4. Implement a function that takes a ROOT pointer and an integer x as input and prints out the number of times x is in the list.
5. Implement a function that takes a ROOT pointer and an integer k as input and prints out the value of the k-th NODE in the list. (consider safety check on the value of k).
6. (*) Implement a function that takes a ROOT pointer and an integer k as input and prints out the value of the k-th NODE from the last NODE in the list (safety check on the value of k).
7. Implement a function that takes a ROOT pointer, two integers k and x as input, creates a NODE with x as its value and then inserts this new NODE as the k-th NODE in the list (safety check on the value of k).
8. Implement a function that takes a ROOT pointer and an integer x as input and removes all the NODE that have x as its value.
9. (**) Implement a function that takes a ROOT pointer as input and removes all the duplicate values in the list. That is, if value x occurs in the list multiple times, keep only one NODE with x and remove all others.

MeshSec Lab

# Homework 5 (* indicates the difficulty level.)

10. (*) Implement a function that takes a ROOT pointer as input and rearranges the list such that the values are sorted in the increasing order.
11. (**) Implement a function that takes a ROOT pointer as input and rearranges the list such that the odd-indexed NODEs appear in front of the even-indexed NODEs. For example, the list 3->5->7->9->8->4->2 will become 3->7->8->2->5->9->4.
12. (**) Implement a function that takes a ROOT pointer as input and reverses the list. For example, the list 3->5->7->9 will become 9->7->5->3.
13. (*) Implement a function that takes two ROOT pointers and returns a ROOT pointer to a new linked list that merges the two input lists in the alternating fashion. For example, if the two input lists are 3->5->7->9 and 8->4, the merged list should be 3->8->5->4->7->9.
14. (***) Implement a function that takes two ROOT pointers P1 and P2, and checks whether P2 is a sub-list of P1. For example, 5->4->7 is a sub-list of 3->8->5->4->7->9.

MeshSec Lab

11

# Homework 5 (* indicates the difficulty level.)

15. (*) Implement a function that takes two ROOT pointers P1 and P2, and removes all the NODEs in P2 from P1. For example, if P1 is 3->8->5->4->8->7->9->2 and P2 is 2->0->8->1, then P1 will become 3->5->4->7->9.
16. (**) Implement a function that takes two ROOT pointers P1 and P2, and appends the reversed version of P2 at the back of P1. For example, if P1 is 3->5->7->9 and P2 is 8->4->9, P1 should become 3->5->7->9->9->4->8.
17. (**) Implement a function that takes two ROOT pointers P1 and P2 to two linked list both sorted in the increasing order, and returns a ROOT pointer to a merged linked list that is also sorted in the increasing order. For example, if P1 is 3->5->7->9 and P2 is 4->9, the new list should become 3->4->5->7->9->9.
18. (**) Implement a function that takes a ROOT pointer as input and checks whether there is a pair of NODEs where the value of one NODE is a multiple of the other. Return the "index" of the NODE with the smaller value and 0 if no such pair. For example, on 3->8->4->7->9->2, you can return either 3 (for 4) or 6 (for 2).

MeshSec Lab